

DATA STRUCTURES & ALGORITHMS LAB

PRACTICAL FILE



SCHOOL OF COMPUTER SCIENCE

UNIVERSITY OF PETROLEUM AND ENERGY STUDIES
DEHRADUN (INDIA), 248007

SUBMITTED TO:

Dr. Soumen Das

SUBMITTED BY:

NAME: Aakash Yadav

SAP ID: 590020652

BRANCH: AIML

INDEX:

S.No.	Objective	Date	Sign.
1.	1. Maximum Subarray Sum - Kadane's Algorithm Given an integer array arr[], find the subarray (containing at least one element) which has the maximum possible sum, and return that sum 2. Maximum consecutive ones after flipping zeroes • Input Array • Constant value=k (the no. of zeroes you can flip)	15/08/25	
2.	1. Create a linked list by taking input from the user. 2. Find the middle of the linked list without using the length function. 3. Perform the deletion operation at the beginning of the linked list	03/09/25	
3.	Reverse a linked list • Entire linked list • Reversing the linked list using given positions of start and stop.	03/09/25	
4.	1. Given a string s containing three types of brackets {}, () and []. Determine whether the Expression are balanced or not. An expression is balanced if each opening bracket has a corresponding closing bracket of the same type, the pairs are properly ordered and no bracket closes before its matching opening bracket. 2. Given a stack with push(), pop(), and empty() operations, The task is to delete the middle element of it without using any additional data structure. 3. Implement Stack operations using Linked List.	24/09/25	
5.	1. Implement queue operations using Array. 2. Implement queue operations using Linked list. 3. Implement circular queue using array and linked list	27/09/25	
6.	1. Implement Quick sort algorithm. Find execution time for already sorted order input and unsorted input.	01/10/25	

7.	2. Implement Linear search algorithm. 3. Implement binary search algorithm.	01/10/25	
8.	1. Represent tree data structure with array 2. Represent Tree data structure with linked list 3. apply pre, in, post, level order traversal to display 4. search an element from a binary tree	29/10/25	
9.	Q1. Take random inputs from user and build a Max heap. Q2. Implement the concept of Heap sort algorithm.	19/11/25	

LAB ACTIVITY (Exp-01)

1. Maximum Subarray Sum - Kadane's Algorithm

Given an integer array arr[], find the subarray (containing at least one element) which has the maximum possible sum, and return that sum

Brute Force Method:-

```
#include <stdio.h>
#include <limits.h>

void findMaxSubarrayBruteForce(int arr[], int n) {
    int maxSum = INT_MIN;
    int startIdx = -1;
    int endIdx = -1;

    for (int i = 0; i < n; i++) {
        int currentSum = 0;

        for (int j = i; j < n; j++) {
            currentSum += arr[j];

            if (currentSum > maxSum) {
                maxSum = currentSum;
                startIdx = i;
                endIdx = j;
            }
        }
    }

    printf("Maximum Subarray Sum: %d\n", maxSum);
    printf("Subarray: [");
    for (int k = startIdx; k <= endIdx; k++) {
        printf("%d", arr[k]);
        if (k < endIdx) {
            printf(", ");
        }
    }
    printf("]\n");
}

int main() {
    int n;
    printf("Enter the length of array: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter the values of array");
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
}
```

```

    findMaxSubarrayBruteForce(arr, n);

    return 0;
}

```

```

PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab5(2409)> cd "Data structures and algorithm\lab1(0608)" ; if ($?) { gcc maxsubarray.c -o maxsubarray }
Enter the length of array: 7
Enter the values of array: 2 3 -8 7 -1 2 3
Maximum Subarray Sum: 11
Subarray: [7, -1, 2, 3]
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab1(0608)> cd "Data structures and algorithm\lab1(0608)" ; if ($?) { gcc maxsubarray.c -o maxsubarray }
Enter the length of array: 2
Enter the values of array: -2 -4
Maximum Subarray Sum: -2
Subarray: [-2]
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab1(0608)> cd "Data structures and algorithm\lab1(0608)" ; if ($?) { gcc maxsubarray.c -o maxsubarray }
Enter the length of array: 5
Enter the values of array: 5 4 1 7 8
Maximum Subarray Sum: 25
Subarray: [5, 4, 1, 7, 8]
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab1(0608)>

```

Dynamic Method:-

```

#include<stdio.h>

void main() {
    int n;
    printf("Enter the length of array: ");
    scanf("%d",&n);

    int arr[n],i;
    printf("Enter the values of array");
    for(i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
}

```

```

int max_c= arr[0];
int max_g=arr[0];
int start=0,end=0,s=0;
for(i=1;i<n;i++){
    if(arr[i]>(max_c+arr[i])){
        max_c=arr[i];
        s=i;
    }
    else
        max_c=max_c+arr[i];
    if(max_c>max_g){
        max_g=max_c;
        start=s;
        end=i;
    }
}
printf("Maximum Subarray Sum: %d\n", max_g);
printf("Subarray: [");
for (int k = start; k <= end; k++) {
    printf("%d", arr[k]);
    if (k < end) {
        printf(", ");
    }
}
printf("]\n");
}

```

```

PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab1(0608)> cd "
tructures and algorithm\lab1(0608)" ; if ($?) { gcc kadane.c -o kadane
Enter the length of array: 7
Enter the values of array 2 3 -8 7 -1 2 3
Maximum Subarray Sum: 11
Subarray: [7, -1, 2, 3]
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab1(0608)> cd "
tructures and algorithm\lab1(0608)" ; if ($?) { gcc kadane.c -o kadane
Enter the length of array: 2
Enter the values of array -2 -4
Maximum Subarray Sum: -2
Subarray: [-2]
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab1(0608)> cd "
tructures and algorithm\lab1(0608)" ; if ($?) { gcc kadane.c -o kadane
Enter the length of array: 5
Enter the values of array 5 4 1 7 8
Maximum Subarray Sum: 25
Subarray: [5, 4, 1, 7, 8]
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab1(0608)>

```

2. Maximum consecutive ones after flipping zeroes

- Input Array
- Constant value=k (the no. of zeroes you can flip)

```
#include<stdio.h>
#include<stdlib.h>

void main(){
    int n;
    printf("Enter the length of array: ");
    scanf("%d",&n);

    int arr[n],i;
    printf("Enter the values of array");
    for(i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    int k;
    printf("Enter the value of k: ");
    scanf("%d",&k);

    int left = 0;
    int right = 0;
    int zeroCount = 0;
    int maxLen = 0;

    while (right < n) {
        if (arr[right] == 0) {
            zeroCount++;
        }

        while (zeroCount > k) {
            if (arr[left] == 0) {
                zeroCount--;
            }
            left++;
        }

        int currentWindowLength = right - left + 1;
        if (currentWindowLength > maxLen) {
            maxLen = currentWindowLength;
        }

        right++;
    }

    printf("Maximum consecutive ones after flipping at most %d zeros: %d\n", k, maxLen);
}
```

```
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab1(0608)> cd "d:\Data structures and algorithm\lab2(1308)\\" ; if ($?) { gcc maxconsecones.c -o maxconsecones }
Enter the length of array: 11
Enter the values of array 1 1 1 0 0 0 1 1 1 1 0
Enter the value of k: 2
Maximum consecutive ones after flipping at most 2 zeros: 6
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab2(1308)> cd "d:\Data structures and algorithm\lab2(1308)\\" ; if ($?) { gcc maxconsecones.c -o maxconsecones }
Enter the length of array: 7
Enter the values of array 1 0 0 1 0 1 1
Enter the value of k: 2
Maximum consecutive ones after flipping at most 2 zeros: 5
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab2(1308)> |
```

LAB ACTIVITY (Exp-2&3)

1. Create a linked list by taking input from the user.
2. Find the middle of the linked list without using the length function.
3. Perform the deletion operation at the beginning of the linked list

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* next;
} node;

node* head = NULL;

void createnode() {
    node *q, *temp;
    temp = (node*)malloc(sizeof(node));
    printf("Enter data:");
    scanf("%d", &temp->data);
    temp->next = NULL;

    if (head == NULL) {
        head = temp;
    } else {
        q = head;
        while (q->next != NULL) {
            q = q->next;
        }
        q->next = temp;
    }
}

void display(node* h) {
    int k=1;
    while (h != NULL) {
        printf("Data %d: %d\n", k, h->data);
        h = h->next;
        k++;
    }
}

void displayMiddle(node *head){
    node *slow,*fast;
    slow=fast=head;
    if(head==NULL){
        printf("The list is empty");
    }
    while(fast!=NULL && fast->next!=NULL){
        fast=fast->next->next;
        slow=slow->next;
    }
    printf("\nMiddle data:%d",slow->data);
}
```

```

}

node* deleteatBegin(node *head){
    if (head == NULL) {
        printf("\nList is already empty!\n");
        return NULL;
    }
    node *temp = head;
    head = head->next;
    free(temp);
    return head;
}

int main() {
    int n, i;
    printf("Enter the number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        createnode();
    }

    printf("Linked list contents:\n");
    display(head);
    displayMiddle(head);
    head = deleteatBegin(head);
    printf("\nNew list after deletion at the beginning:\n");
    display(head);

    return 0;
}

```

```

PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab2(1308)> cd "d:\Data structures and algorithm\lab3(0309)" ; if ($?) { gcc createlinked.c -o createlinked }
Enter the number of nodes: 5
Enter data:10
Enter data:20
Enter data:30
Enter data:40
Enter data:50
Linked list contents:
Data 1: 10
Data 2: 20
Data 3: 30
Data 4: 40
Data 5: 50

Middle data:30
New list after deletion at the beginning:
Data 1: 20
Data 2: 30
Data 3: 40
Data 4: 50
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab3(0309)>

```

4. Reverse a linked list

- Entire linked list
- Reversing the linked list using given positions of start and stop

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* next;
} node;

node* head = NULL;

void createnode() {
    node *q, *temp;
    temp = (node*)malloc(sizeof(node));
    printf("Enter data:");
    scanf("%d", &temp->data);
    temp->next = NULL;

    if (head == NULL) {
        head = temp;
    } else {
        q = head;
        while (q->next != NULL) {
            q = q->next;
        }
        q->next = temp;
    }
}

void display(node* h) {
    int k=1;
    while (h != NULL) {
        printf("Data %d: %d\n", k, h->data);
        h = h->next;
        k++;
    }
}

node * reverseLinkedList(node *head){
    node *pnode, *cnode, *nnode;
    pnode=nnode=NULL;
    cnode=head;
    while(cnode)
    {
        nnode=cnode->next;
        cnode->next=pnode;
        pnode=cnode;
        cnode=nnode;
    }
    return pnode;
}
```

```

}

node* reverseLinkedListPos(node* head, int m, int n) {
    node *pnode = NULL, *cnode = head, *nnode = NULL;
    node *tempnode1 = NULL, *tempnode2 = NULL;
    int k = 1;

    if (m == 1) {
        tempnode1 = NULL;
        tempnode2 = cnode;
    } else {
        while (k < m) {
            tempnode1 = cnode;
            cnode = cnode->next;
            k++;
        }
        tempnode2 = cnode;
    }

    pnode = NULL;
    while (cnode && m <= n) {
        nnode = cnode->next;
        cnode->next = pnode;
        pnode = cnode;
        cnode = nnode;
        m++;
    }

    if (tempnode1)
        tempnode1->next = pnode;
    else
        head = pnode;

    tempnode2->next = cnode;

    return head;
}

int main() {
    int n, i;
    printf("Enter the number of nodes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        createnode();
    }

    printf("Linked list contents:\n");
    display(head);

    head=reverseLinkedList(head);
    printf("Reversed Linked List:\n");
    display(head);

    int m, o;
    printf("Enter the positions you want to reverse the Linked List:");
    scanf("%d %d", &m, &o);
    head=reverseLinkedListPos(head, m, o);
}

```

```
printf("Reversed Linked List:\n");  
display(head);  
  
return 0;  
}
```

```
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab3(0309)> cd "d:\S  
tructures and algorithm\lab4(1009)\\" ; if ($?) { gcc reverselinkedlist.c -o  
($?) { .\reverselinkedlist }  
Enter the number of nodes: 5  
Enter data:10  
Enter data:20  
Enter data:30  
Enter data:40  
Enter data:50  
Linked list contents:  
Data 1: 10  
Data 2: 20  
Data 3: 30  
Data 4: 40  
Data 5: 50  
Reversed Linked List:  
Data 1: 50  
Data 2: 40  
Data 3: 30  
Data 4: 20  
Data 5: 10  
Enter the positions you want to reverse the Linked List:2  
4  
Reversed Linked List:  
Data 1: 50  
Data 2: 20  
Data 3: 30  
Data 4: 40  
Data 5: 10  
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab4(1009)> |
```

LAB ACTIVITY (Exp-04)

1. Given a string *s* containing three types of brackets {}, () and []. Determine whether the Expression are balanced not. An expression is balanced if each opening bracket has a corresponding closing bracket of the same type, the pairs are properly ordered and no bracket closes before its matching opening bracket.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <string.h>

#define MAX 100

char stack[MAX];
int top = -1;

void push(char c) {
    if (top == MAX - 1) {
        printf("Stack overflow\n");
        exit(1);
    }
    stack[++top] = c;
}

char pop() {
    if (top == -1) {
        printf("Stack underflow\n");
        exit(1);
    }
    return stack[top--];
}

bool isEmpty() {
    return top == -1;
}

bool isBalanced(char *exp) {
    for (int i = 0; exp[i] != '\0'; i++) {
        char ch = exp[i];

        if (ch == '(' || ch == '{' || ch == '[') {
            push(ch);
        }
        else if (ch == ')' || ch == '}' || ch == ']') {
            if (isEmpty()) return false;

            char topChar = pop();

            if ((ch == ')' && topChar != '(') ||
                (ch == '}' && topChar != '{') ||
                (ch == ']' && topChar != '['))
                return false;
        }
    }

    return isEmpty();
}
```

```

        (ch == ']' && topChar != '[')) {
            return false;
        }
    }
}

return isEmpty();
}

int main() {
    char exp[MAX];

    printf("Enter an expression: ");
    scanf("%s", exp);

    if (isBalanced(exp)) {
        printf("Balanced\n");
    } else {
        printf("Not Balanced\n");
    }

    return 0;
}

```

```

PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab5(2409)> cd "Data structures and algorithm\lab5(2409)" ; if ($?) { gcc balanced.c -o balanced.exe }
Enter an expression: [{()}]
Balanced
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab5(2409)> cd "Data structures and algorithm\lab5(2409)" ; if ($?) { gcc balanced.c -o balanced.exe }
Enter an expression: ([{}])
Not Balanced
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab5(2409)>

```

2. Given a stack with push(), pop(), and empty() operations, The task is to delete the middle element of it without using any additional data structure.

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

int stack[MAX];
int top = -1;

void push(int x) {
    if (top == MAX - 1) {
        printf("Stack Overflow\n");
        return;
    }
    stack[++top] = x;
}

int pop() {
    if (top == -1) {
        printf("Stack Underflow\n");
        return -1;
    }
    return stack[top--];
}

int isEmpty() {
    return top == -1;
}

void deleteMiddleUntil(int current, int size) {
    if (isEmpty() || current == size) {
        return;
    }

    int x = pop();

    if (current != size / 2) {
        deleteMiddleUntil(current + 1, size);
        push(x);
    } else {
        deleteMiddleUntil(current + 1, size);
    }
}

void deleteMiddle() {
    int size = top + 1;
    deleteMiddleUntil(0, size);
}

void printStack() {
    for (int i = top; i >= 0; i--) {
        printf("%d ", stack[i]);
    }
    printf("\n");
}
```

```

}

int main() {
    int n, val;

    printf("Enter number of elements in stack: ");
    scanf("%d", &n);

    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        push(val);
    }

    printf("\nOriginal Stack (Top to Bottom): ");
    printStack();

    deleteMiddle();

    printf("After Deleting Middle Element: ");
    printStack();

    return 0;
}

```

```

PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab5(2409)> cd "d:
tructures and algorithm\lab5(2409)\\" ; if ($?) { gcc delmiddle.c -o delmid
le }
Enter number of elements in stack: 5
Enter 5 elements: 10 20 30 40 50

Original Stack (Top to Bottom): 50 40 30 20 10
After Deleting Middle Element: 50 40 20 10
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab5(2409)> cd "d:
tructures and algorithm\lab5(2409)\\" ; if ($?) { gcc delmiddle.c -o delmid
le }
Enter number of elements in stack: 10
Enter 10 elements: 5 9 6 7 6 6 5 10 12 9

Original Stack (Top to Bottom): 9 12 10 5 6 6 7 6 9 5
After Deleting Middle Element: 9 12 10 5 6 7 6 9 5
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab5(2409)>

```

3. Implement Stack operations using Linked List.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* top = NULL;

void push(int x) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = x;
    newNode->next = top;
    top = newNode;
}

int pop() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return -1;
    }
    struct Node* temp = top;
    int popped = temp->data;
    top = top->next;
    free(temp);
    return popped;
}

int peek() {
    if (top == NULL) {
        printf("Stack is empty\n");
        return -1;
    }
    return top->data;
}

void display() {
    struct Node* temp = top;
    if (temp == NULL) {
        printf("Stack is empty\n");
        return;
    }
    printf("Stack (Top to Bottom): [");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("]\n");
}

int main() {
    int n, val;
```

```

printf("Enter number of elements in stack: ");
scanf("%d", &n);

printf("Enter %d elements:\n", n);
for (int i = 0; i < n; i++) {
    scanf("%d", &val);
    push(val);
}

printf("\nStack created successfully!\n");
display();

printf("Top element is %d\n", peek());

printf("Popped element is %d\n", pop());
display();

return 0;
}

```

```

PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab5(2409)> cd
tructures and algorithm\lab5(2409)\\" ; if ($?) { gcc stacklinkedlist.c
{ .\stacklinkedlist }
Enter number of elements in stack: 5
Enter 5 elements:
10 20 30 40 50

Stack created successfully!
Stack (Top to Bottom): [50 40 30 20 10 ]
Top element is 50
Popped element is 50
Stack (Top to Bottom): [40 30 20 10 ]
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab5(2409)>

```

LAB ACTIVITY (Exp-05)

1. Implement queue operations using Array.

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

void enqueue(int x) {
    if (rear == MAX - 1) {
        printf("Queue Overflow\n");
        return;
    }
    if (front == -1) front = 0;
    queue[++rear] = x;
    printf("%d enqueued\n", x);
}

void dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow\n");
        return;
    }
    printf("%d dequeued\n", queue[front++]);
}

void display() {
    if (front == -1 || front > rear) {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue elements: ");
    for (int i = front; i <= rear; i++)
        printf("%d ", queue[i]);
    printf("\n");
}

int main() {
    int choice, value;
    while (1) {
        printf("\n--- Queue using Array ---\n");
        printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
```

```
switch (choice) {  
    case 1:  
        printf("Enter value to enqueue: ");  
        scanf("%d", &value);  
        enqueue(value);  
        break;  
    case 2:  
        dequeue();  
        break;  
    case 3:  
        display();  
        break;  
    case 4:  
        exit(0);  
    default:  
        printf("Invalid choice!\n");  
}  
}  
return 0;  
}
```

```
--- Queue using Array ---  
1. Enqueue  
2. Dequeue  
3. Display  
4. Exit  
Enter your choice: 1  
Enter value to enqueue: 10  
10 enqueued
```

```
--- Queue using Array ---  
1. Enqueue  
2. Dequeue  
3. Display  
4. Exit  
Enter your choice: 1  
Enter value to enqueue: 20  
20 enqueued
```

```
--- Queue using Array ---  
1. Enqueue  
2. Dequeue  
3. Display  
4. Exit  
Enter your choice: 1  
Enter value to enqueue: 30  
30 enqueued
```

```
--- Queue using Array ---  
1. Enqueue  
2. Dequeue  
3. Display  
4. Exit  
Enter your choice: 2  
10 dequeued
```

```
--- Queue using Array ---  
1. Enqueue  
2. Dequeue  
3. Display  
4. Exit  
Enter your choice: 3  
Queue elements: 20 30
```

2. Implement queue operations using Linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* front = NULL;
struct Node* rear = NULL;

void enqueue(int x) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->data = x;
    temp->next = NULL;

    if (rear == NULL) {
        front = rear = temp;
    } else {
        rear->next = temp;
        rear = temp;
    }
    printf("%d enqueued\n", x);
}

void dequeue() {
    if (front == NULL) {
        printf("Queue Underflow\n");
        return;
    }
    struct Node* temp = front;
    printf("%d dequeued\n", front->data);
    front = front->next;
    if (front == NULL) rear = NULL;
    free(temp);
}

void display() {
    if (front == NULL) {
        printf("Queue is empty\n");
        return;
    }
    struct Node* temp = front;
    printf("Queue elements: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

```
int main() {
    int choice, value;
    while (1) {
        printf("\n--- Queue using Linked List ---\n");
        printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &value);
                enqueue(value);
                break;
            case 2:
                dequeue();
                break;
            case 3:
                display();
                break;
            case 4:
                exit(0);
            default:
                printf("Invalid choice!\n");
        }
    }
    return 0;
}
```

--- Queue using Linked List ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter value to enqueue: 10

10 enqueued

--- Queue using Linked List ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter value to enqueue: 20

20 enqueued

--- Queue using Linked List ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter value to enqueue: 30

30 enqueued

--- Queue using Linked List ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 2

10 dequeued

--- Queue using Linked List ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 3

Queue elements: 20 30

--- Queue using Linked List ---

1. Enqueue

3. Implement circular queue using array and linked list

1. Using Array

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

void enqueue(int x) {
    if ((rear + 1) % MAX == front) {
        printf("Circular Queue Overflow\n");
        return;
    }
    if (front == -1) {
        front = rear = 0;
    } else {
        rear = (rear + 1) % MAX;
    }
    queue[rear] = x;
    printf("%d enqueued\n", x);
}

void dequeue() {
    if (front == -1) {
        printf("Circular Queue Underflow\n");
        return;
    }
    int x = queue[front];
    if (front == rear) {
        front = rear = -1;
    } else {
        front = (front + 1) % MAX;
    }
    printf("%d dequeued\n", x);
}

void display() {
    if (front == -1) {
        printf("Circular Queue is empty\n");
        return;
    }
    printf("Circular Queue elements: ");
    int i = front;
    while (1) {
        printf("%d ", queue[i]);
        if (i == rear) break;
        i = (i + 1) % MAX;
    }
    printf("\n");
}
```

```
int main() {
    int choice, value;
    while (1) {
        printf("\n--- Circular Queue using Array ---\n");
        printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &value);
                enqueue(value);
                break;
            case 2:
                dequeue();
                break;
            case 3:
                display();
                break;
            case 4:
                exit(0);
            default:
                printf("Invalid choice!\n");
        }
    }
    return 0;
}
```

--- Circular Queue using Array ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter value to enqueue: 10

10 enqueued

--- Circular Queue using Array ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter value to enqueue: 20

20 enqueued

--- Circular Queue using Array ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter value to enqueue: 30

30 enqueued

--- Circular Queue using Array ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 2

10 dequeued

--- Circular Queue using Array ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 3

Circular Queue elements: 20 30

--- Circular Queue using Array ---

1. Enqueue
2. Dequeue

2. Using Linked List

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* front = NULL;
struct Node* rear = NULL;

int size = 0;
int N = 5;

void enqueue(int x) {
    if (size == N) {
        printf("Circular Queue Overflow\n");
        return;
    }

    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->data = x;
    temp->next = NULL;

    if (front == NULL) {
        front = rear = temp;
        rear->next = front;
    } else {
        rear->next = temp;
        rear = temp;
        rear->next = front;
    }

    size = (size + 1) % (N + 1);
    printf("%d enqueued\n", x);
}

void dequeue() {
    if (front == NULL) {
        printf("Circular Queue Underflow\n");
        return;
    }

    int x;
    if (front == rear) {
```

```

        x = front->data;
        free(front);
        front = rear = NULL;
    } else {
        struct Node* temp = front;
        x = temp->data;
        front = front->next;
        rear->next = front;
        free(temp);
    }

    size--;
    printf("%d dequeued\n", x);
}

void display() {
    if (front == NULL) {
        printf("Circular Queue is empty\n");
        return;
    }

    printf("Circular Queue elements: ");
    struct Node* temp = front;
    do {
        printf("%d ", temp->data);
        temp = temp->next;
    } while (temp != front);
    printf("\n");
}

int main() {
    int choice, value;
    while (1) {
        printf("\n--- Circular Queue using Linked List ---\n");
        printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &value);
                enqueue(value);
                break;
            case 2:
                dequeue();
                break;

```

```
        case 3:
            display();
            break;
        case 4:
            exit(0);
        default:
            printf("Invalid choice!\n");
    }
}
return 0;
}
```

--- Circular Queue using Linked List ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter value to enqueue: 10

10 enqueued

--- Circular Queue using Linked List ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter value to enqueue: 20

20 enqueued

--- Circular Queue using Linked List ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter value to enqueue: 30

30 enqueued

--- Circular Queue using Linked List ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 2

10 dequeued

--- Circular Queue using Linked List ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 3

Circular Queue elements: 20 30

--- Circular Queue using Linked List ---

1. Enqueue

LAB ACTIVITY (Exp-06&07)

1. Implement Quick sort algorithm. Find execution time for already sorted order input and unsorted input.

```
#include <stdio.h>

// Function to swap two numbers
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Partition function
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // last element as pivot
    int i = (low - 1);
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

// QuickSort function
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    quickSort(arr, 0, n - 1);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
}
```

```

}
printf("\n");

return 0;
}

```

```

PS D:\SEMESTER I\LAB WORK\Data structures and algorithm> cd "d:\SEMESTER I\LAB WORK\Data structures and algorithm\lab7(0110)" ; if ($?) { gcc quicksort.c -o quicksort }
Enter number of elements: 8
Enter 8 elements: 12 4 2 11 9 10 5 4
Sorted array: 2 4 4 5 9 10 11 12
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab7(0110)> cd "d:\SEMESTER I\LAB WORK\Data structures and algorithm\lab7(0110)" ; if ($?) { gcc quicksort.c -o quicksort }

```

execution time for already sorted order input:- $O(n^2)$

execution time for unsorted input:- $O(n \log n)$

2. Implement Linear search algorithm.

```

#include <stdio.h>

int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key)
            return i;
    }
    return -1;
}

int main() {
    int n, key;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter element to search: ");
    scanf("%d", &key);

    int result = linearSearch(arr, n, key);
    if (result != -1)
        printf("Element %d found at index %d\n", key, result);
    else
        printf("Element not found\n");

    return 0;
}

```

```

PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab7
tructures and algorithm\lab7(0110)\ " ; if ($?) { gcc linear_
\linear_search }
Enter number of elements: 7
Enter 7 elements: 10 5 7 2 9 11 3
Enter element to search: 7
Element 7 found at index 2
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab7

```

3. Implement binary search algorithm.

```

#include <stdio.h>

int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

int main() {
    int n, key;
    printf("Enter number of elements (sorted order): ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d sorted elements: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter element to search: ");
    scanf("%d", &key);

    int result = binarySearch(arr, n, key);
    if (result != -1)
        printf("Element %d found at index %d\n", key, result);
    else
        printf("Element not found\n");

    return 0;
}

```

Element 7 found at index 2

```
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab7(0110)>
structures and algorithm\lab7(0110)" ; if ($?) { gcc binary_search.c
\binary_search }
```

Enter number of elements (sorted order): 8

Enter 8 sorted elements: 2 4 8 10 45 78 98 102

Enter element to search: 98

Element 98 found at index 6

```
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab7(0110)>
```

LAB ACTIVITY (Exp-08)

Q1) Represent tree data structure with array

```
#include <stdio.h>
#define MAX 50

int tree[MAX];
int n;

void inputTree() {
    printf("Enter number of nodes in the binary tree: ");
    scanf("%d", &n);

    printf("Enter %d node values (level-wise):\n", n);
    for (int i = 0; i < n; i++) {
        printf("Node %d: ", i);
        scanf("%d", &tree[i]);
    }
}

void displayTree() {
    printf("\nBinary Tree (Array Representation):\n");
    for (int i = 0; i < n; i++) {
        printf("Index %d-->Data:- %d\n", i, tree[i]);
    }
}

void showRelations() {
    printf("\nParent-Child Relationships:\n");
    for (int i = 0; i < n; i++) {
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        printf("Parent: %d ", tree[i]);
        if (left < n)
            printf("| Left Child: %d", tree[left]);
        else
            printf("| Left Child: None");

        if (right < n)
            printf(" | Right Child: %d", tree[right]);
        else
            printf(" | Right Child: None");

        printf("\n");
    }
}

int main() {
    inputTree();
    displayTree();
    showRelations();
    return 0;
}
```

```
Enter number of nodes in the binary tree: 6
```

```
Enter 6 node values (level-wise):
```

```
Node 0: 0
```

```
Node 1: 1
```

```
Node 2: 2
```

```
Node 3: 3
```

```
Node 4: 4
```

```
Node 5: 5
```

```
Binary Tree (Array Representation):
```

```
Index 0-->Data:- 0
```

```
Index 1-->Data:- 1
```

```
Index 2-->Data:- 2
```

```
Index 3-->Data:- 3
```

```
Index 4-->Data:- 4
```

```
Index 5-->Data:- 5
```

```
Parent-Child Relationships:
```

```
Parent: 0 | Left Child: 1 | Right Child: 2
```

```
Parent: 1 | Left Child: 3 | Right Child: 4
```

```
Parent: 2 | Left Child: 5 | Right Child: None
```

```
Parent: 3 | Left Child: None | Right Child: None
```

```
Parent: 4 | Left Child: None | Right Child: None
```

```
Parent: 5 | Left Child: None | Right Child: None
```

```
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab9(1510)> █
```

Q2) Represent Tree data structure with linked list

Q3) apply pre, in, post, level order traversal to display

Q4) search an element from a binary tree

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->left = newNode->right = NULL;
    return newNode;
}

struct Node* createTree() {
```

```

int data;
printf("Enter root node data (-1 for no tree): ");
scanf("%d", &data);

if (data == -1)
    return NULL;

struct Node* root = createNode(data);

struct Node* queue[50];
int front = 0, rear = 0;
queue[rear++] = root;

while (front < rear) {
    struct Node* current = queue[front++];

    printf("Enter left child of %d (-1 for no left child): ", current->data);
    scanf("%d", &data);
    if (data != -1) {
        current->left = createNode(data);
        queue[rear++] = current->left;
    }

    printf("Enter right child of %d (-1 for no right child): ", current->data);
    scanf("%d", &data);
    if (data != -1) {
        current->right = createNode(data);
        queue[rear++] = current->right;
    }
}

return root;
}

void levelOrder(struct Node* root) {
    if (root == NULL) {
        printf("Tree is empty.\n");
        return;
    }

    struct Node* queue[50];
    int front = 0, rear = 0;

    queue[rear++] = root;

    printf("Level Order Traversal: ");
    while (front < rear) {
        struct Node* current = queue[front++];
        printf("%d ", current->data);

        if (current->left)
            queue[rear++] = current->left;
        if (current->right)
            queue[rear++] = current->right;
    }
}

void preorder(struct Node* root) {
    if (root == NULL) return;

```

```

    printf("%d ", root->data);
    preorder(root->left);
    preorder(root->right);
}

void inorder(struct Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}

void postorder(struct Node* root) {
    if (root == NULL) return;
    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->data);
}

int search(struct Node* root, int key) {
    if (root == NULL)
        return 0;

    if (root->data == key)
        return 1;

    return search(root->left, key) || search(root->right, key);
}

int main() {
    struct Node* root = createTree();
    printf("\n");
    levelOrder(root);

    printf("\nPreorder Traversal: ");
    preorder(root);

    printf("\nInorder Traversal: ");
    inorder(root);

    printf("\nPostorder Traversal: ");
    postorder(root);

    int key;
    printf("\n\nEnter element to search: ");
    scanf("%d", &key);

    if (search(root, key))
        printf("Element %d found in the tree.\n", key);
    else
        printf("Element %d not found in the tree.\n", key);

    return 0;
}

```

```
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab9(1510)> cd "d:\SEMESTER I\LAB WORK\Data structures and algorithm\lab9(1510)"
($?) { gcc tree_linked.c -o tree_linked } ; if ($?) { .\tree_linked }
Enter root node data (-1 for no tree): 0
Enter left child of 0 (-1 for no left child): 1
Enter right child of 0 (-1 for no right child): 2
Enter left child of 1 (-1 for no left child): 3
Enter right child of 1 (-1 for no right child): 4
Enter left child of 2 (-1 for no left child): 5
Enter right child of 2 (-1 for no right child): 6
Enter left child of 3 (-1 for no left child): -1
Enter right child of 3 (-1 for no right child): -1
Enter left child of 4 (-1 for no left child): -1
Enter right child of 4 (-1 for no right child): -1
Enter left child of 5 (-1 for no left child): -1
Enter right child of 5 (-1 for no right child): -1
Enter left child of 6 (-1 for no left child): -1
Enter right child of 6 (-1 for no right child): -1

Level Order Traversal: 0 1 2 3 4 5 6
Preorder Traversal: 0 1 3 4 2 5 6
Inorder Traversal: 3 1 4 0 5 2 6
Postorder Traversal: 3 4 1 5 6 2 0

Enter element to search: 5
Element 5 found in the tree.
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab9(1510)>
```

LAB ACTIVITY (Exp-09)

Q1. Take random inputs from user and build a Max heap.

Q2. Implement the concept of Heap sort algorithm.

```
#include <stdio.h>
#include <stdlib.h>

void swap(int *a, int *b) {
    int t = *a; *a = *b; *b = t;
}

void heapify(int arr[], int heapSize, int i) {
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if (left < heapSize && arr[left] > arr[largest])
        largest = left;
    if (right < heapSize && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, heapSize, largest);
    }
}

void buildMaxHeap(int arr[], int n) {
    for (int i = n/2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

void heapSort(int arr[], int n) {
    buildMaxHeap(arr, n);
    for (int i = n-1; i > 0; i--) {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
```

```
int *arr = malloc(n * sizeof(int));

printf("Enter %d random numbers:\n", n);
for (int i = 0; i < n; i++)
    scanf("%d", &arr[i]);

buildMaxHeap(arr, n);
printf("\nArray after building Max-Heap:\n");
printArray(arr, n);

heapSort(arr, n);
printf("\nArray after Heap Sort:\n");
printArray(arr, n);

free(arr);
return 0;
}
```

```
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm> cd "d:\SEMESTER I\LAB
max_heap.c -o max_heap } ; if ($?) { .\max_heap }
Enter number of elements: 7
Enter 7 random numbers:
1 4 8 6 3 2 9

Array after building Max-Heap:
9 6 8 4 3 2 1

Array after Heap Sort:
1 2 3 4 6 8 9
PS D:\SEMESTER I\LAB WORK\Data structures and algorithm\lab10(1911)> |
```